Refinement-Guided Behavioral Program Synthesis

Integrating Z3 with Djex and Leant

Design objective

Extend type-directed term synthesis from "find an inhabitant of this type" to "find a well-typed term that satisfies this behavioral contract", while preserving Djex's logical/operational distinctions and Leant's kernel-checked trust boundary.

Date: 10 August 2026; updated 15 August 2026
Repositories: VladimirReshetnikov/Djex
VladimirReshetnikov/Leant
z3prover/z3

This report is pinned to exact repository revisions listed in Section 3. It proposes compatible incremental changes, concrete APIs, algorithms, encodings, soundness labels, tests, and an implementation sequence.

Status, 15 August 2026: the level-1 layer proposed here has since been implemented in Djex and Leant as the finite-list-spine `Length` contract dialect: checked contracts and assumed provider laws, canonical `QF_LIA` queries, a capability-probed live Z3 worker, independent counterexample replay, bounded input-box validation, and a binary-product extension, exact derived-domain ranking through strict relational Natural quotient consequences, and sealed descriptor-bound Z3 launch, reachable from `:synth` behind the explicit `--length-ranking-config` opt-in. One deliberate divergence from the proposal: verified candidates are stably reordered, never pruned, and raw solver status carries no authority. Levels 2 and 3 remain future work; the dated reports under `docs/reports/` in both repositories record the exact boundaries. The successor proposal, *Precise Behavioral Code Synthesis in Leant and Djex with Z3* (`docs/Z3_Behavioral_Synthesis_Proposal/`, 15 August 2026), starts from that implemented layer and lays out those next increments.

Status, 19 August 2026: the opt-in semantic post-filter proposed as level 1 has also shipped: `:synth --behavior-mode filter` performs command-authorized hard Length filtering in which only an independently replayed counterexample rejects a candidate and every other assessment retains it, supported by command-local counterexample banks and progressive same-run filter batching; the default rank lane still reorders without pruning. Levels 2 and 3 (typed sketch completion and semantic branch pruning) remain future work.

Contents

1 Executive summary **1**

2 Implementation addendum: exact root-quotient domains **2**

3 Repository snapshot and architectural findings **4**

3.1 Djex: the right neutral insertion point already exists 4

3.2 The generated AST is useful but currently too weakly typed 5

3.3 Exference already has the data needed for deep semantic pruning 6

3.4 Leant already has the strongest possible final checker 6

3.5 Z3 capabilities that map directly to this problem 6

4 The proposed capability: behavioral contracts **7**

4.1 A contract is more than a Boolean expression 7

4.2 Contracts should be elaborated in the source language 8

4.3 The central verification query is existential 8

4.4 Semantic summaries are a checked inventory, not annotations sprinkled on terms . 8

4.5 Use products of abstractions 9

5 Architecture **9**

5.1 Dependency direction 9

5.2 New neutral modules 10

5.3 Retaining typed generated terms 11

5.4 Do not overload existing evidence 11

5.5 Completeness and trust are a vector 12

6 Algorithms **12**

6.1 Mode 1: streaming semantic post-filtering 12

6.2 Mode 2: counterexample-guided typed sketch synthesis 13

6.2.1 Why this is preferable to generic syntax synthesis in Z3 14

6.3 Mode 3: semantic pruning of Exference search nodes 14

6.3.1 Oracle call control 15

6.4 Optimize-based ranking 15

6.5 Fixedpoint/PDR for recursive programs 16

7 Worked example: a polymorphic list transformer preserving length **16**

7.1 Semantic statement 16

7.2 What Z3 says about common candidates 16

7.3 Minimal SMT-LIB verification . . . 17
7.4 A first typed grammar . . . 17
7.5 Adding bag preservation . . . 18
7.6 Leant form of the same query . . . 18

8 Djex integration in detail **18**
8.1 Public API shape . . . 18
8.2 Session construction and inventory fingerprints . . . 19
8.3 Source-level contract syntax for Djex . . . 20
8.4 Residual constraints and dictionary semantics . . . 20
8.5 Preserving laziness and cancellation . . . 20
8.6 Semantic deduplication before rendering . . . 21

9 Leant integration in detail **21**
9.1 Elaborate the contract in Lean . . . 21
9.2 Retain structured candidates in the engine . . . 22
9.3 Verification pipeline . . . 22
9.4 Proof-plan reconstruction tiers . . . 23
9.5 User-facing commands . . . 23
9.6 Proof-producing synthesis as a first-class feature . . . 23

10 High-value features unlocked by the same integration **24**
10.1 Minimal counterexamples for every rejected candidate . . . 24
10.2 Unsat cores for impossible specifications . . . 24
10.3 Semantic equivalence clustering . . . 24
10.4 Round-trip adapter and serializer synthesis . . . 25
10.5 Lens and update-law synthesis . . . 25
10.6 Bounds-safe arithmetic and bit-vector code . . . 25
10.7 Protocol transition and state-machine synthesis . . . 26
10.8 Contract inference . . . 26
10.9 Precondition and invariant abduction . . . 26
10.10 Contract-aware provider ranking . . . 26

11 Soundness and semantic boundaries **27**
11.1 Haskell is not a total set-theoretic language . . . 27
11.1.1 Recommended Haskell modes . . . 27
11.2 Abstract interpretation can be sound without being complete . . . 28
11.3 Trust in provider summaries . . . 28

11.4 Z3 statuses must remain three-valued . . . 28
11.5 What an “unsat” result proves . . . 29
11.6 Leant’s trust story . . . 29
11.7 No-solution claims . . . 29

12 Z3 integration engineering **30**
12.1 Subprocess first, C API second . . . 30
12.2 Robust SMT-LIB session protocol . . . 30
12.3 C API ownership model . . . 31
12.4 Symbol encoding . . . 31
12.5 Normalization and caching . . . 31
12.6 Determinism and reproducibility . . . 32
12.7 Performance tactics . . . 32

13 Implementation sequence **33**
13.1 Stage A: semantic verification MVP . . . 33
13.2 Stage B: Leant proof-grade contracts . . . 33
13.3 Stage C: semantic selection and explanation . . . 33
13.4 Stage D: typed sketch CEGIS . . . 33
13.5 Stage E: Exference semantic oracle . . . 34
13.6 Stage F: recursive invariant synthesis . . . 34
13.7 Concrete file-level changes . . . 34

14 Testing and evaluation **35**
14.1 Unit and property tests . . . 35
14.2 Differential tests . . . 35
14.3 Metamorphic tests . . . 35
14.4 Regression corpus . . . 36
14.5 Performance measurements . . . 36

15 Risks and design traps **37**

16 Final recommendation **38**

A Proposed Haskell API in more detail **38**
A.1 Semantic IR . . . 38
A.2 Checked contracts . . . 39
A.3 Symbolic interpretation . . . 39
A.4 Verdicts and counterexamples . . . 40

A.5 Sketches 40

B Complete SMT-LIB sketch for the length example **41**

C Generated Lean checker examples **42**

C.1 Definitional candidate 42

C.2 Summary-theorem candidate 42

C.3 Structural recursive candidate 42

C.4 Arithmetic residual closed by omega 43

D Theory and feature matrix **43**

E Source map **44**

F One-page implementation checklist **45**

1 Executive summary

Primary recommendation

Build a *refinement-guided synthesis layer* around Djex. Djinn and Exference continue to generate type-correct proof terms; a new typed semantic layer translates supported candidates and contracts into solver obligations; Z3 filters, explains, completes, and eventually prunes the search; Leant re-elaborates the program and, when a proof-grade result is requested, kernel-checks a Lean theorem establishing the contract. Z3 is an untrusted semantic search oracle in Leant, not an extension of the trusted kernel.

The most valuable first feature is exactly the one suggested in the request:

```
:synth ([a] -> [a]) where length result == length input
```

but it should be implemented as the first instance of a general contract system, not as a one-off list-length filter. The same machinery can support size preservation, permutation laws, sortedness, arithmetic bounds, bit-vector behavior, round trips, lens laws, state-machine protocols, and recursive invariants.

The proposed integration has three increasing levels of power:

1. **Semantic post-filtering.** Run existing Djinn/Exference search unchanged, symbolically summarize each complete candidate, and retain only candidates for which Z3 cannot find a counterexample. This is the lowest-risk MVP.
2. **Typed sketch completion.** Ask Djex to construct a finite typed grammar or a partial expression with holes. Encode each hole as a finite selector and let Z3 choose constructors, providers, constants, and branch predicates. Verify the decoded candidate by counterexample-guided inductive synthesis (CEGIS).
3. **Semantic branch pruning.** Exference already carries a partial expression with typed holes in every search node. Expose a pure stepping API and place an effectful semantic oracle around it; prune a branch only when its necessary semantic constraints are unsatisfiable. This can change search complexity by orders of magnitude for tightly constrained queries.

The architectural boundary matters. Djex's shared `QueryEvidence` currently says whether the type search found validated candidates or proved non-inhabitation. It must not be overloaded to mean that a behavioral law was proved. Add a separate `SemanticVerdict`; keep search completeness, semantic encoding exactness, provider summary trust, solver status, and final proof checking as independent axes.

For Haskell, a law such as length preservation must state its semantic domain. The initial exact fragment should quantify over finite, spine-total lists and total, structural candidate terms. Bottoms, exceptions, unrestricted recursion, infinite lists, `seq`, unsafe operations, and effects remain outside that theorem unless an explicit bounded operational mode is selected. For Lean, the endpoint can be stronger: Leant can synthesize both a term and a theorem, then ask Lean's kernel to check both.

The highest-leverage deliverables

- A pure, backend-neutral semantic IR and contract checker in Djex.
- A long-lived Z3 SMT-LIB session for the MVP, followed by an optional C-API FFI.

- Typed candidate retention before Djinn/Exference erase private type annotations.
- Streaming constrained selection using Djex's existing monadic result fold.
- A Leant contract serializer that elaborates the contract in Lean and emits a canonical semantic S-expression rather than asking Haskell to parse Lean.
- Counterexamples, unsat cores, semantic equivalence classes, and proof plans as first-class user-facing results.
- A staged path from filtering to CEGIS, Optimize-based minimization, and Fixedpoint/Spacer-based recursive invariant synthesis.

2 Implementation addendum: exact root-quotient domains

The current implemented checkpoint advances Leant's closed live Length-ranking configuration through scalar v19 and product v20. These versions retain the complete v17/v18 sealed descriptor launch, owner-thread-scoped usable-work deadline, cooperative checkpoints, deferred worker opening, non-vacuous preferences, counterexample simplification, and scalar-v5 or pair-v5 contract grammar. They replace only the applicable-domain strategy with `strict-relational-positive-affine-quotient-v1`.

Listing 1: Public Leant root-quotient surface

```
enableLengthRankingStrictRelationalPositiveAffineQuotientApplicableDomainValidation
  :: LengthInputBoxLimits
  -> LengthRankingPolicy
  -> LengthRankingPolicy

StrictRelationalPositiveAffineQuotientApplicableDomainEstablished
LengthSpinePairStrictRelationalPositiveAffineQuotientApplicableDomainEstablished

renderLengthStrictRelationalPositiveAffineQuotientApplicableDomainValidationNote
renderLengthSpinePairStrictRelationalPositiveAffineQuotientApplicableDomainValidationNote

lengthRankingConfigurationFileStrictRelationalPositiveAffineQuotientVersion
  == 19
lengthRankingConfigurationFileSpinePairStrictRelationalPositiveAffineQuotientVersion
  == 20
```

Write $q_d(E)$ for Natural floor quotient E quot d, with $d > 0$. Djex adds exactly four proof-only implications for positive-affine operands:

$$\begin{aligned} q_d(A) \le B &\Longrightarrow A \le dB + (d-1), \\ A \le q_d(B) &\Longrightarrow dA \le B, \\ \neg(q_d(A) \le B) &\Longrightarrow d(B+1) \le A, \\ \neg(A \le q_d(B)) &\Longrightarrow B + 1 \le dA. \end{aligned}$$

A top-level equality contributes the two directed non-strict rules in source order. The inherited synchronous rule-once closure can therefore propagate a later literal ceiling through either orientation. For example, $x \le q_3(y)$ together with $y \le 8$ derives source-ordered maxima $[2, 8]$; $q_3(x) = y$ together with $x \le 1$ derives $y \le 0$.

Deliberately narrow authority

Exactly one relation side may contain a quotient, and that quotient must be at the operand root. A nested quotient, a quotient embedded below a sum, scale, monus, minimum, maximum, modulo, or conditional, quotients at both relation roots, an unsupported dividend or opposite operand, and an unsupported whole formula grant no coverage rule or partial bound. Quotient-free clauses delegate to the strict predecessor exactly. Unsupported clauses remain in the actual precondition replay if other clauses establish a complete box.

Validation retains the predecessor's precedence: reject input width before demanding the precondition or evaluation limits; scan clauses and close rules; report the first compact missing input; validate maxima and Cartesian cardinality; then replay assignments canonically. Only complete query-owned replay releases an establishment receipt. The runtime route remains newest-first four-entry MRU replay, derived-domain replay, canonical origin replay, live Z3 query, and optional post-unsat explicit box. An inapplicable quotient pass is a pure miss; a counterexample crosses the existing query-owned simplification and MRU seams.

Versions	Applicable-domain rule	Usable work	Launch
v1–v6	none	none; eager historical route	pathname/direct
v7/v8	positive affine	none; deferred historical route	pathname/direct
v9/v10	positive affine	runtime-unscoped v1	pathname/direct
v11/v12	relational positive affine	none; deferred historical route	pathname/direct
v13/v14	relational positive affine	owner-thread scoped/checkpointed v2	pathname/direct
v15/v16	strict relational positive affine	owner-thread scoped/checkpointed v2	pathname/direct
v17/v18	strict relational positive affine	owner-thread scoped/checkpointed v2	sealed descriptor
v19/v20	strict relational root quotient	owner-thread scoped/checkpointed v2	sealed descriptor

The scalar-only compatibility decoder still rejects v4–v20. The generalized decoder reaches v19/v20 only after the exact v1–v18 cascade returns its closed unsupported- version sentinel; v21 is unsupported. Complete scalar-v19 and pair-v20 JSON documents, including the inherited descriptor execution member `executableLaunch = descriptor-bound-executable-v1`, scoped-v2 budget, deferred opening, and contract grammars, are recorded in `README.md` and `docs/reports/2026-08-15-strict-relational-positive-affine-quotient-length-ranking.md`.

Evidence and identity remain separate

The four rewrites emit no SMT-LIB command and consume no solver status. They do not alter normalized contract bytes, checked problem identity, sealed query fingerprints or wire bytes, protocol, process, ready-worker, scalar-run, or pair-run identity. Djex adds only nominal scalar and product quotient-establishment receipt tags; Leant adds the matching nominal assessments and renderers, with no canonical evidence bytes of its own. The receipt remains model/provider-relative finite-spine evidence: it is not a Lean theorem, source-language totality or effect claim, solver-soundness claim, or pruning authority.

The descriptor guarantee remains main-image-only. It binds the configured digest to the sealed staged executable bytes, not to an interpreter, dynamic loader, shared library, file capability, solver

semantics, or solver result. An all-pure deferred batch still opens no executable descriptor and no worker.

At the quiescent v19/v20 characterization checkpoint, `test-unit/Spec.hs` contains 23,334 lines and 443 exact `testCase` occurrences. The focused quotient-consequence group contains seven passing cases under the strict test build. These counts describe this dated snapshot, not a stable API contract.

The corresponding Djex extraction report is `lib/Djex/docs/reports/2026-08-15-strict-relational-positive-affine-quotient-length-applicable-domain.md`. This checkpoint does not declare Boolean/DNF extraction, or any other behavioral extension, to be definitively next.

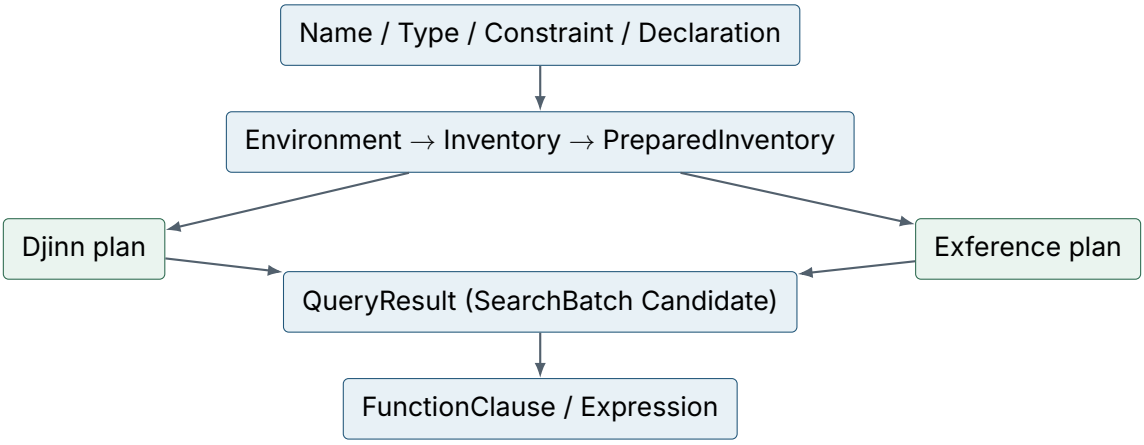
3 Repository snapshot and architectural findings

The analysis was performed against the following exact heads on 10 August 2026. Pinning matters because both Djex and Leant are evolving rapidly and Leant's Djex submodule follows concrete integration work rather than a released package boundary.

Repository	Pinned revision	Relevant state
Djex	`f50b5eac8672`	Shared parser-independent synthesis foundation; checked Djinn and Exference adapters; structural higher-kinded provider assignments; scope-checked generated syntax.
Leant	`0160237e4bdf`	Djex-backed Lean 4 REPL; goal serializer; structural synthesis engine; live provider inventory; every rendered candidate re-elaborated by Lean before presentation.
Z3	`0bd679a404be`	SMT solver with models, unsat cores, incremental solving, algebraic datatypes, sequences, arrays, bit-vectors, Optimize, recursive functions, and Fixedpoint/PDR.

3.1 Djex: the right neutral insertion point already exists

Djex is not merely a command-line wrapper around two historical engines. Its `synthesis/src/Language/Haskell/Synthesis` subtree is a deliberately backend-neutral library. The current pipeline is approximately:



The most important existing properties are these:

- `Query.hs` separates logical evidence from operational progress. A candidate discovered before a budget is exhausted remains a valid candidate; a truncated empty search is not silently promoted to non-inhabitation.
- `Search.hs` makes truncation reasons explicit and intentionally makes no logical claim.
- `Candidate.hs` combines checked generated output, residual type-class constraints, and backend-specific details without erasing the latter.
- `Generated.hs` supplies a common scope-aware expression AST, holes, alpha equivalence, substitution, bottom-up rewriting, simplification, global reference discovery, and expression size.
- `Selection.hs` already contains a monadic, batch-by-batch fold over a lazy result trace. This is exactly the primitive needed to run an effectful solver filter without forcing the rest of Exference's stream.
- `Declaration.hs` treats neutral declarations as the authority and backend caches/ratings as projections. Semantic summaries should follow the same pattern.

The checked Djinn adapter returns one terminal result batch. The checked Exference adapter returns a lazy sequence of batches. Both expose shared candidates, but they retain backend-specific metrics and evidence. This is a strong base for adding a third, orthogonal concern: semantic contracts.

3.2 The generated AST is useful but currently too weakly typed

The shared generated expression contains precisely the constructs needed for symbolic interpretation:

Listing 2: Simplified shape of Djex's shared generated expression

```
data Expression local
  = Local local
  | Global Name
  | Lambda [Pattern local] (Expression local)
  | Apply (Expression local) (Expression local)
  | VisibleTypeApplication (Expression local) VisibleTypeArgument
  | Tuple [Expression local]
  | Hole local
  | Let (Pattern local) (Expression local) (Expression local)
  | Case (Expression local) [(Pattern local, Expression local)]
```

However, this is the *checked output* representation after the engines have erased most private type annotations. A semantic interpreter cannot correctly translate an arbitrary application, overloaded global, polymorphic instantiation, or case arm from this tree alone. The integration therefore needs one of two safe boundaries:

1. an initial first-order re-elaborator that reconstructs types from the goal and the exact inventory, refusing anything ambiguous; or
2. preferably, a new typed projection emitted at the same point where Djinn's proof term and Exference's independently checked expression become the shared generated tree.

The second option is the durable design. It preserves the existing public AST and renderers while attaching a type to every node or stable node path.

3.3 Exference already has the data needed for deep semantic pruning

Internally, Exference represents every queued branch as a `SearchNode` containing:

- a sequence of typed goals, each associated with an expression hole;
- the partial typed expression being constructed;
- lexical scopes and type substitutions;
- provider schemes, deconstructors, constraints, depth, and ranking state.

The root starts with a single expression hole. Introduction, tuple, provider, application, let, and case steps fill one hole and allocate further typed holes. This is almost exactly a typed sketch state. The obstacle is architectural rather than representational: the search core is pure. Calling Z3 through `unsafePerformIO` would break cancellation, determinism, exception boundaries, and referential transparency. The proper refactor is to expose pure branch expansion and let an outer effectful driver ask a semantic oracle whether a branch is still feasible.

3.4 Leant already has the strongest possible final checker

Leant's `Leant.Synth.Engine` is deliberately the only module importing Djex. It translates a Lean goal fragment, invokes Djinn and/or Exference, renders candidate variants, and returns a bounded ranked stream. In `Main.hs`, each variant is then submitted to the active Lean session as a temporary example. A candidate is shown only if elaboration succeeds, no fatal response occurs, and no `sorry` is present. The generated checker has the shape:

Listing 3: Current Leant candidate checking boundary

```
set_option autoImplicit true in
noncomputable example : (goal) := (candidate)
```

This is an unusually favorable trust architecture. Z3 can be used aggressively for search and pruning because a final Leant result need not trust Z3 for type correctness or for the behavioral theorem. Leant can generate a theorem statement and proof plan, then ask Lean's kernel to check it.

A necessary representation change is that `SynthOutcome` currently returns rendered strings rather than structured expressions. Semantic filtering should occur before that erasure, or `SynthCandidate` should retain both the Djex expression and its rendered Lean variants.

3.5 Z3 capabilities that map directly to this problem

The pinned Z3 tree exposes all required capabilities through the C API and most language bindings. The command-line executable also accepts SMT-LIB 2 from standard input and exposes wall-clock/memory controls, making it a practical zero-FFI MVP.

Z3 facility	Use in Djex/Leant
Incremental solver, push/pop	Reuse declaration and contract context across many candidates and CEGIS iterations.
Models and model evaluation	Produce concrete counterexamples, selector assignments, and synthesized constants.
Unsat cores	Explain mutually inconsistent behavioral clauses or infeasible sketch choices.

Z3 facility	Use in Djex/Leant
Optimize	Minimize AST cost, provider cost, case count, literal magnitude, or contract relaxation lexicographically.
Algebraic datatypes	Exact symbolic models for admitted finite inductive data.
Sequences and regular expressions	Lists/strings/bytes and regular-language constraints where sequence semantics is preferable to measures.
Arrays and EUF	State maps, finite-memory protocols, bags/count maps, and opaque pure providers.
Bit-vectors	Exact machine arithmetic, masks, packing, bounds, and overflow behavior.
Recursive function definitions	Definitional summaries for a controlled structural fragment.
Fixedpoint/PDR (Spacer)	Horn-clause verification, recursive invariants, protocol reachability, and ranking/invariant synthesis.
Proof objects	Optional diagnostics or future independently checked certificates; not the recommended Leant trust boundary.
Interrupt and timeout APIs	Cancellation aligned with Djex/Leant command deadlines.

The design should not depend on a SyGuS front end. Ordinary SMT, finite selector variables, models, blocking clauses, Optimize, and CEGIS are sufficient and give Djex control over the typed grammar. A repository-wide search of the pinned Z3 revision did not locate `synth-fun` or `check-synth` command handling; keeping synthesis in Djex also avoids coupling the public feature to any future parser-specific support.

4 The proposed capability: behavioral contracts

4.1 A contract is more than a Boolean expression

A useful contract object must record the quantified program inputs, precondition, postcondition, semantic domain, admissible abstraction, and proof policy. The following is representative, not final syntax:

Listing 4: Backend-neutral contract vocabulary

```
data Contract ty variable = Contract
  { contractInputs       :: [Binder ty variable]
  , contractPrecondition :: Formula variable
  , contractResult       :: Binder ty variable
  , contractPostcondition :: Formula variable
  , contractDomain       :: SemanticDomain
  , contractMode         :: VerificationMode
  , contractObjectives   :: [Objective]
  }

data SemanticDomain
  = MathematicalTotal
  | FiniteSpineTotal
  | BoundedOperational Bounds

data VerificationMode
  = RequireProof
```

```
  | AcceptSolverUnsat
  | AcceptBoundedValidation Bounds
```

The checked form should be opaque. Construction validates that every variable is bound, every operator is sorted, the result type agrees with the type query, all referenced measures are available, and the selected semantic domain admits every provider the search may use.

4.2 Contracts should be elaborated in the source language

Neither Djex nor Z3 should parse arbitrary Lean terms. Likewise, the solver layer should not guess the meaning of arbitrary Haskell source text.

- A Haskell/Djex frontend parses a compact contract DSL and resolves names through the same exact inventory used by synthesis.
- Leant asks Lean to elaborate both the type and the contract. A Lean metaprogram serializes the elaborated semantic fragment into a canonical S-expression, just as the current goal serializer already does for types.
- The neutral Djex contract layer receives only checked names, sorts, measures, and formulas. No source-language name resolution leaks into Z3 encoding.

This arrangement avoids a class of subtle bugs involving namespace aliases, implicit arguments, overloaded notation, coercions, reducibility, and universes.

4.3 The central verification query is existential

Suppose a candidate denotes a total function f and the contract is

$$\forall x.\ P(x) \Rightarrow Q(x, f(x)).$$

Do not ask Z3 to prove this universal formula directly. Ask it for a counterexample:

$$\exists x.\ P(x) \wedge \neg Q(x, f(x)).$$

The program inputs are declared as free constants. Then:

- `sat` gives a concrete counterexample model;
- `unsat` establishes the contract relative to the exact encoding and asserted semantic summaries;
- `unknown` remains unknown, with Z3's reason retained.

This quantifier-free or lightly quantified shape is usually much more robust than a universal theorem. It also fits CEGIS naturally.

4.4 Semantic summaries are a checked inventory, not annotations sprinkled on terms

Add a separate `SemanticInventory` keyed by Djex's exact structural `Name`. Each entry states what a provider means in one or more abstraction domains:

Listing 5: Provider summary sketch

```
data ProviderSummary = ProviderSummary
```

```
  { summaryProvider :: Name
  , summaryScheme   :: CheckedSemanticScheme
  , summaryClauses  :: [SummaryClause]
  , summaryTrust    :: SummaryTrust
  , summaryOrigin   :: SummaryOrigin
  }

data SummaryTrust
  = DerivedFromDatatypeDeclaration
  | LeanKernelChecked TheoremName
  | BuiltinAxiom SemanticModelVersion
  | UserAssumed SourceLocation
  | ObservedOnly Bounds
```

Constructor summaries can be derived mechanically from a checked `DataTypeDeclaration`. For example, a list constructor has length summary $\text{len}(\texttt{Cons}\ x\ xs) = 1 + \text{len}(xs)$. A Lean provider summary should normally be admitted only when a theorem with the expected shape is kernel-checked. Haskell summaries may initially be builtin or explicitly assumed; the result must expose that trust.

Do not overload `Declaration`'s opaque annotation field with semantic meaning. Annotations are source metadata; semantic summaries have versioning, proof origin, and domain applicability that deserve their own checked index.

4.5 Use products of abstractions

No single encoding is best for all contracts. A candidate may be interpreted in a product of independently selected domains:

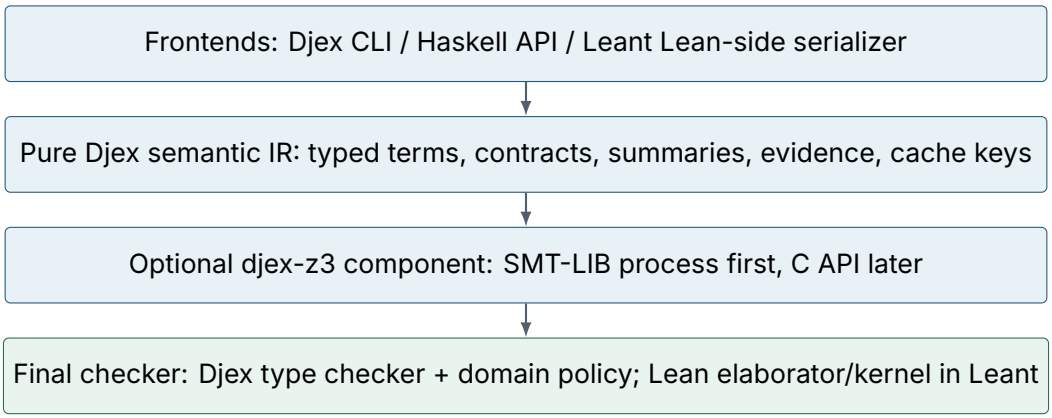
- **Measure domain:** length, size, height, constructor counts, numeric bounds.
- **Sequence domain:** element order and concatenation.
- **Bag/provenance domain:** multiplicities or element-origin tokens.
- **Scalar domain:** integers, rationals, bit-vectors, Booleans.
- **Shape domain:** datatype constructor/tag information.
- **State domain:** arrays/maps and protocol state.
- **Opaque domain:** uninterpreted functions with congruence only.

The contract compiler requests only the domains it needs, and every provider must have a compatible summary in each requested domain. This keeps a length-preservation query in quantifier-free linear integer arithmetic instead of unnecessarily encoding entire lists.

5 Architecture

5.1 Dependency direction

The clean architecture is a four-layer stack:



The neutral semantic IR belongs close to Djex's existing synthesis foundation and must remain pure. Solver process management and FFI handles belong in a separate optional component. This preserves lightweight use of Djex when no Z3 executable or library is available and prevents solver state from contaminating pure search code.

5.2 New neutral modules

A plausible module layout is:

```
Language.Haskell.Synthesis.TypedGenerated
Language.Haskell.Synthesis.Semantic.Sort
Language.Haskell.Synthesis.Semantic.Term
Language.Haskell.Synthesis.Semantic.Formula
Language.Haskell.Synthesis.Semantic.Contract
Language.Haskell.Synthesis.Semantic.Measure
Language.Haskell.Synthesis.Semantic.Summary
Language.Haskell.Synthesis.Semantic.Inventory
Language.Haskell.Synthesis.Semantic.Elaborate
Language.Haskell.Synthesis.Semantic.Symbolic
Language.Haskell.Synthesis.Semantic.Evidence
Language.Haskell.Synthesis.Semantic.CacheKey
```

The Z3-specific package or Cabal component can then provide:

```
Language.Haskell.Synthesis.Z3.Session
Language.Haskell.Synthesis.Z3.SExpr
Language.Haskell.Synthesis.Z3.SMTLib
Language.Haskell.Synthesis.Z3.Encode
Language.Haskell.Synthesis.Z3.Model
Language.Haskell.Synthesis.Z3.Optimize
Language.Haskell.Synthesis.Z3.Fixedpoint
Language.Haskell.Synthesis.Z3.Verify
```

Leant-specific code should stay behind its present narrow engine boundary:

```
Leant.Synth.Contract
Leant.Synth.Semantic
Leant.Synth.ProofPlan
Leant.Z3
```

5.3 Retaining typed generated terms

The recommended representation is a checked shared tree with a type at every node, plus a projection to today's `Expression`:

Listing 6: One possible typed output boundary

```
data TypedExpression ty local = TypedExpression
  { typedExpressionType :: !ty
  , typedExpressionNode :: TypedNode ty local
  }

data TypedNode ty local
  = TLocal local
  | TGlobal Name [VisibleTypeArgument]
  | TLambda [(Pattern local, ty)] (TypedExpression ty local)
  | TApply (TypedExpression ty local) (TypedExpression ty local)
  | TTuple [TypedExpression ty local]
  | THole HoleId ty
  | TLet (Pattern local) (TypedExpression ty local)
         (TypedExpression ty local)
  | TCase (TypedExpression ty local)
          [(TypedPattern ty local, TypedExpression ty local)]

eraseTypes :: TypedExpression ty local -> Expression local
```

An alternative, less invasive form is an `ElaboratedCandidate` containing the existing expression and a total map from stable node paths to types. The tree form is harder to desynchronize and is better for sketch synthesis; the side-table form may be useful as a compatibility bridge.

Typed output should be produced before backend erasure:

- in Djinn, while projecting the proof term to generated syntax;
- in Exference, from the independently checked internal expression after type substitutions and constraint resolution, before projecting to shared syntax.

The existing candidate and renderer APIs remain source-compatible. New constrained APIs can request typed output explicitly.

5.4 Do not overload existing evidence

Use an orthogonal wrapper:

Listing 7: Separate type-search and semantic evidence

```
data SemanticVerdict
  = ContractProved ProofBasis
  | ContractRefuted Counterexample
  | ContractValidatedWithin Bounds ValidationSummary
  | ContractUnknown UnknownReason

data ProofBasis
  = LeanKernelChecked TheoremName
  | Z3Unsat SolverFingerprint EncodingFingerprint
  | CertificateChecked CertificateFingerprint
  | ExactNormalization SemanticModelVersion

data ConstrainedCandidate candidate = ConstrainedCandidate
```

```
  { constrainedCandidate :: candidate
  , constrainedVerdict   :: SemanticVerdict
  , constrainedCost      :: SemanticCost
  }

data ConstrainedResult metadata candidate = ConstrainedResult
  { typeSearchResult :: QueryResult metadata candidate
  , semanticResults  :: [ConstrainedCandidate candidate]
  , semanticProgress :: SolverProgress
  }
```

This prevents several invalid inferences. In particular:

- Djinn can prove that a type is inhabited while every inspected inhabitant fails a behavioral contract.
- Exference can be truncated after every inspected candidate fails; this is not proof that no satisfying term exists.
- Z3 can return `unsat` for a candidate under assumed provider summaries; this is not the same as a Lean-kernel theorem.
- A finite grammar can be exhausted semantically even though the unrestricted type has other inhabitants outside the grammar.

5.5 Completeness and trust are a vector

Every constrained synthesis answer should carry at least these independent dimensions:

Dimension	Representative values
Type-search status	complete, candidate-bounded, step-bounded, queue-pruned, identifier-space exhausted.
Grammar status	unrestricted calculus, exact finite grammar, size-bounded grammar, provider-bounded grammar.
Semantic translation	exact for admitted fragment, abstraction-sound but incomplete, bounded execution, unsupported.
Provider trust	derived constructor, Lean theorem, builtin axiom, user assumption, observed sample.
Solver status	sat with model, unsat, unknown with reason, timeout, interrupted, protocol failure.
Final checking	Lean kernel theorem, source-language typecheck only, independently checked certificate, solver-only, testing-only.

A concise UI can summarize this vector, but the structured API must not collapse it.

6 Algorithms

6.1 Mode 1: streaming semantic post-filtering

This mode can be implemented without changing either search calculus. It is the right first milestone because it establishes contract elaboration, semantic summaries, Z3 transport, evidence, counterexamples, caching, cancellation, and UI semantics before any search-internal changes.

For each complete candidate:

1. obtain or reconstruct its typed generated expression;
2. select the semantic domains required by the contract;
3. symbolically interpret the expression using exact constructor semantics and checked provider summaries;
4. assert the negated postcondition under the precondition;
5. ask Z3 for satisfiability;
6. retain `unsat` candidates, report models for `sat`, and preserve `unknown` according to caller policy.

Pseudocode:

Listing 8: Streaming post-filtering over existing result batches

```
filterConstrained
  :: SolverSession
  -> CheckedContract
  -> SemanticInventory
  -> [QueryResult metadata candidate]
  -> IO (Maybe Progress, ConstrainedAccumulator candidate)
filterConstrained solver contract inventory =
  foldAllQueryResultsM admissible inspect emptyAccumulator
 where
  admissible = candidateHasNoUnsupportedResiduals

  inspect acc candidate = do
    typed <- requireTypedCandidate candidate
    case symbolicInterpret inventory contract typed of
      Left unsupported ->
        pure (recordUnknown candidate unsupported acc)
      Right interpretation -> do
        verdict <- verifyCounterexampleQuery solver contract interpretation
        pure (record candidate verdict acc)
```

The use of Djex's existing `foldAllQueryResultsM` is important. It preserves Exference's laziness, permits short-circuiting after the requested number of satisfying candidates, and does not retain the root of a large result trace. Djinn's single terminal batch passes through the same API.

A useful result-selection policy is not merely "keep all unsat candidates". It should support:

- first proof-grade candidate;
- best k candidates under a combined syntactic/semantic cost;
- first candidate in each semantic equivalence class;
- proof-grade candidates preferred, bounded candidates as fallback;
- stop after n solver checks without improvement.

6.2 Mode 2: counterexample-guided typed sketch synthesis

Post-filtering wastes work when a behavioral contract is selective. The next step is to let Z3 choose among a finite, type-correct set of construction alternatives. Djex remains responsible for the typed grammar; Z3 never invents an untyped program.

A typed hole $h_i : \tau_i$ receives alternatives $e_{i,0}, \ldots, e_{i,k_i-1}$ generated from the lexical context, constructors, and provider inventory. Introduce an integer or bit-vector selector s_i with $0 \leq s_i < k_i$. The symbolic value of the hole is:

$$[\![h_i]\!] = \text{ite}(s_i = 0, [\![e_{i,0}]\!], \text{ite}(s_i = 1, [\![e_{i,1}]\!], \ldots)).$$

The first implementation should use an acyclic grammar and a strict AST-size bound. Recursive calls can be admitted later under structural decrease and CHC verification. The synthesis loop is CEGIS:

Listing 9: Typed CEGIS loop

```
cegis :: Sketch -> CheckedContract -> IO SynthesisOutcome
cegis sketch contract = loop initialExamples
 where
  loop examples = do
    model <- solveSelectors sketch contract examples
    case model of
      Unsat -> pure NoProgramInGrammar
      Unknown why -> pure (SynthesisUnknown why)
      Sat assignment -> do
        let candidate = decodeSketch sketch assignment
        counterexample <- verifyCandidate contract candidate
        case counterexample of
          Unsat -> pure (Found candidate)
          Sat input -> loop (input : examples)
          Unknown why -> pure (CandidateUnknown candidate why)
```

The synthesis query constrains the candidate on a growing finite example set. The verification query asks existentially for a fresh counterexample over the full admitted symbolic domain. Blocking only the complete selector vector is correct but slow; learned clauses should instead block the smallest responsible subassignment when possible.

6.2.1 Why this is preferable to generic syntax synthesis in Z3

- The grammar is produced from Djex's exact type environment, including local scopes, constructors, higher-rank boundaries, visible type applications, and provider instantiations.
- Every decoded model denotes a well-scoped, type-correct candidate by construction.
- Djex can retain its own term-size and provider-rating heuristics.
- The encoding can exploit domain-specific summaries without exposing Haskell or Lean syntax to Z3.
- Grammar exhaustion has a precise, reportable meaning.

6.3 Mode 3: semantic pruning of Exference search nodes

Exference is particularly suitable for incremental semantic pruning because each search node already contains a partial expression and a set of typed holes. The semantic oracle should compute a *necessary* condition for some completion of the node to satisfy the contract. If that condition is unsatisfiable, the branch is safe to prune. A satisfiable result is only feasibility evidence, not proof that a completion exists.

A clean refactor is:

Listing 10: Effectful driver around a pure search step

```
data SearchExpansion node candidate
  = Solved candidate
  | Expanded [node]
  | DeadEnd

stepExference :: SearchNode -> SearchExpansion SearchNode Candidate

class Monad m => SemanticOracle m node where
  classifyNode :: CheckedContract -> node -> m BranchVerdict

data BranchVerdict
  = DefinitelyInfeasible UnsatExplanation
  | PossiblyFeasible SemanticLowerBound
  | OracleUnknown UnknownReason
```

The outer driver pops a node, calls `stepExference`, and batches semantic queries for children. Only `DefinitelyInfeasible` removes a child. The lower bound may be added to Exference's priority to rank branches that are semantically closer to the contract.

The abstraction for a hole must be permissive. For example, if a hole of list type can be filled only from a finite grammar whose possible length transforms are $\{n, 0, n+1\}$, then the branch can use the disjunction. If no finite grammar is fixed, the hole's length must be an unconstrained nonnegative integer; otherwise pruning could be unsound.

6.3.1 Oracle call control

A solver call per branch would overwhelm small searches. Use:

- checks only after semantic milestones (provider choice, constructor choice, branch predicate, recursive call), not every administrative transition;
- batched sibling checks with assumptions and unsat cores;
- a cheap abstract interpreter before Z3;
- memoization by alpha-normalized partial term, normalized type substitution, contract, and semantic inventory fingerprint;
- adaptive checking: increase solver frequency only after the queue exceeds a threshold or the contract rejects a high fraction of completed candidates.

6.4 Optimize-based ranking

Z3 Optimize should not replace Djex's mature search ranking at first. It is most useful inside a finite sketch or after candidate generation. A lexicographic objective can be:

$$(\text{proof grade},\ \text{AST size},\ \text{provider cost},\ \text{case count},\ \text{literal magnitude},\ \text{semantic distance}).$$

Hard constraints establish typing and behavior. Soft constraints prefer the original input, avoid unsafe/assumed summaries, preserve all arguments, or use highly rated providers. The model must be decoded and rechecked by the ordinary solver because optimization plus quantified or recursive constraints may return `unknown` or non-obvious bounds.

6.5 Fixedpoint/PDR for recursive programs

Recursive synthesis should not begin by expanding recursive functions into bounded traces and calling that proof. Z3's Fixedpoint API accepts universal Horn clauses and supports PDR-style queries with background assertions. This maps naturally to:

- relational semantics of a recursive function;
- a safety predicate representing the behavioral law;
- unknown inductive invariants from a template family;
- ranking functions or structural decrease conditions.

For a recursive list transformer, define a relation $F(xs, ys)$, constructor rules for the program, and a bad relation Bad(xs, ys) when the contract fails. Query reachability of `Bad`. Fixedpoint `false` establishes safety in the CHC encoding; the answer or cover can help reconstruct an invariant. In Leant, that invariant becomes a hint for an induction proof, not a trusted theorem by itself.

7 Worked example: a polymorphic list transformer preserving length

7.1 Semantic statement

The surface request is:

```
f :: [a] -> [a]
-- Contract: for every admitted xs,
--           length (f xs) = length xs
```

For Haskell, the first proof-grade semantic mode should define "admitted" as:

- `xs` has a finite, total spine;
- element payloads need not be evaluated by a length-only contract;
- the candidate belongs to the total structural fragment and cannot use bottom, exceptions, `seq`, unsafe casts, effects, or unrestricted recursion;
- every referenced provider has a compatible total summary.

Let $n = \text{len}(xs)$ with $n \geq 0$. A length summary maps the candidate to an integer term $L_f(n)$. Verification asks whether

$$n \geq 0 \wedge L_f(n) \neq n$$

is satisfiable.

7.2 What Z3 says about common candidates

Candidate shape	Length summary	Counterexample query
`\xs -> xs`	n	Unsatisfiable.
`\_ -> []`	0	$n = 1$ is a model.

Candidate shape	Length summary	Counterexample query
`tail` with totalized empty case	$\mathrm{ite}(n = 0, 0, n - 1)$	$n = 1$ is a model.
`\xs -> xs ++ xs`	$2n$	$n = 1$ is a model.
`reverse`	n under a checked summary	Unsatisfiable.
`drop k`	$\max(n - k, 0)$	For fixed $k > 0$, $n = 1$ is usually a model; only $k = 0$ works universally.
`take k`	$\min(k, n)$	$n = k + 1$ is a model for finite k.
`\(x:xs) -> xs ++ [x]` with empty case	n	Unsatisfiable in the length domain.

The last row is deliberately important: length preservation is not identity. The contract admits reverse, rotations, arbitrary permutations, and even some programs that drop and duplicate elements in a balanced way. This is a feature, not a bug. A user who wants a permutation should add bag equality; a user who wants identity should state extensional equality.

7.3 Minimal SMT-LIB verification

For a candidate whose summary is $2n$:

Listing 11: Counterexample query for duplicating the list

```
(set-logic QF_LIA)
(set-option :produce-models true)
(declare-const n Int)
(assert (>= n 0))
(define-fun outLen () Int (* 2 n))
(assert (not (= outLen n)))
(check-sat)
(get-value (n outLen))
```

A conforming response includes a model such as $n = 1$, `outLen` $= 2$. For the identity summary, the final assertion simplifies to $n \neq n$ and the result is `unsat`.

The actual session should not emit a new declaration context for every candidate. It should push a candidate scope, assert the candidate-specific definition and bad condition, check, obtain model/core if applicable, then pop.

7.4 A first typed grammar

For a deliberately small grammar over `xs :: [a]`:

$$\begin{aligned} E_{[a]} ::= {} & xs \mid [] \mid \mathrm{reverse}(E_{[a]}) \mid E_{[a]} +\!\!+ E_{[a]} \\ & \mid \mathrm{tail0}(E_{[a]}) \mid \mathrm{take}(E_{\mathbb{N}}, E_{[a]}) \mid \mathrm{drop}(E_{\mathbb{N}}, E_{[a]}) \\ E_{\mathbb{N}} ::= {} & 0 \mid 1 \mid n \mid E_{\mathbb{N}} + E_{\mathbb{N}}. \end{aligned}$$

Every constructor has a length transfer function. Z3 can select a term of bounded size, constrain its transfer function to equal n on accumulated examples, and then verify the decoded term for all $n \geq 0$. Optimize minimizes size and provider cost.

A first synthesis result is likely identity because it has minimum size. Add a novelty constraint to request a nontrivial solution:

$$\exists xs.\ f(xs) \neq xs,$$

or ban the direct variable alternative. The next minimal candidate may be reverse or a single rotation, depending on the inventory and cost model.

7.5 Adding bag preservation

Length plus bag equality gives a permutation law:

$$\text{len}(f(xs)) = \text{len}(xs) \quad \wedge \quad \text{bag}(f(xs)) = \text{bag}(xs).$$

For a bounded element universe, a bag can be an array from element tokens to integer counts. For an unbounded parametric universe, use provenance tokens introduced for input positions and constrain constructors/providers by their token transfer. In the pure parametric structural fragment, this is closely related to a free theorem, but the implementation should not silently assume parametricity when providers may use `seq`, type reflection, unsafe features, or effects.

A sequence-domain encoding can prove order-sensitive laws directly, but measure plus provenance domains are often much cheaper and provide better counterexamples.

7.6 Leant form of the same query

A possible surface syntax is:

```
:synth (f : List alpha -> List alpha)
  where forall xs, (f xs).length = xs.length
```

The displayed ASCII spelling is only illustrative; the real parser should let Lean elaborate ordinary Lean syntax, including implicit parameters and notation. A successful proof-grade result can be presented as:

```
it1 : List alpha -> List alpha := fun xs => xs
it1_spec : forall xs, (it1 xs).length = xs.length := by
  intro xs
  rfl
```

For a rotation or recursive transformer, the generated proof plan may use induction, `simp`, and `omega`. The theorem, not Z3's `unsat`, is the final trusted artifact.

8 Djex integration in detail

8.1 Public API shape

The constrained API should be additive. Existing calls retain their exact semantics. One possible high-level interface is:

Listing 12: Additive constrained-query API

```
data ConstrainedRequest ty options = ConstrainedRequest
  { constrainedTypeRequest :: QueryRequest ty options
  , constrainedContract    :: CheckedContract
```

```
  , constrainedPolicy      :: ConstrainedPolicy
  }

data ConstrainedPolicy = ConstrainedPolicy
  { semanticUnknownPolicy :: UnknownPolicy
  , maximumSolverChecks   :: Maybe Natural
  , requiredProofGrade    :: ProofGrade
  , semanticSelection     :: SemanticSelection
  }

runDjinnConstrained
  :: Z3Session
  -> SemanticInventory
  -> DjinnSession
  -> ConstrainedRequest DjinnType QueryOptions
  -> IO (Either Diagnostic DjinnConstrainedResult)

runExferenceConstrained
  :: Z3Session
  -> SemanticInventory
  -> ExferenceSession
  -> ConstrainedRequest ExferenceType ExferenceOptions
  -> IO (Either Diagnostic [ExferenceConstrainedResult])
```

An even cleaner API separates search from verification so applications can reuse a candidate stream or substitute another solver backend:

```
verifyCandidate
  :: MonadSemanticSolver m
  => CheckedSemanticContext
  -> TypedCandidate
  -> m SemanticVerdict

foldConstrainedResultsM
  :: Monad m
  => (candidate -> m SemanticVerdict)
  -> ConstrainedFoldPolicy
  -> [QueryResult metadata candidate]
  -> m (ConstrainedSelection candidate)
```

The Z3 package then instantiates `MonadSemanticSolver`; tests can use a mock or a small exhaustive evaluator.

8.2 Session construction and inventory fingerprints

A checked semantic session should seal:

- the exact neutral declaration inventory;
- normalized provider summaries and their proof origins;
- datatype-derived measures;
- semantic model version;
- solver capabilities/version;
- namespace mapping from Djex names to collision-free SMT symbols.

Like current Djinn/Exference sessions, this moves fallible validation out of the lazy query result. A malformed summary, unsupported recursive measure, or conflicting name must fail before `Right` exposes a result stream.

A fingerprint should cover structural content, not rendered spelling. It belongs in cache keys and result evidence so a result cannot be replayed after a provider theorem or semantic axiom changes.

8.3 Source-level contract syntax for Djex

A minimal CLI syntax might be:

```
lengthPreserving ? f :: [a] -> [a]
  where forall xs. length (f xs) == length xs
```

or a command-oriented form:

```
:contract f :: [a] -> [a]
  ensures length result == length arg0
:solve f
```

The parser should lower names such as `length` to checked semantic measure IDs, not ordinary executable globals. Start with a compact first-order grammar:

- Boolean connectives and equality/order;
- integer and bit-vector arithmetic;
- named measures on arguments/result;
- constructor/tag tests;
- array/sequence operations admitted by selected domains;
- explicit quantification over finite auxiliary domains only.

Do not start by embedding arbitrary Haskell expressions in contracts. That would immediately inherit bottoms, type classes, laziness, desugaring, and module-resolution complexity.

8.4 Residual constraints and dictionary semantics

Djex candidates may retain type-class obligations. A constrained query must decide whether they are:

- fully resolved before semantic interpretation;
- admitted as exact provider assignments with semantic summaries; or
- rejected/marked unsupported.

Treating a residual class constraint as an uninterpreted Boolean assumption is usually unsound for behavioral laws because the selected dictionary controls method behavior. The semantic inventory must attach summaries to the exact resolved provider or to a class law that is valid for every admitted instance.

8.5 Preserving laziness and cancellation

The constrained layer is effectful, but the candidate list may remain lazy. Use a producer/consumer discipline:

- force only enough of the next candidate to type/semantically elaborate it;
- check candidates under a bounded worker pool, preserving deterministic result order unless the caller explicitly selects fastest-first mode;
- propagate asynchronous exceptions; do not convert them to semantic unknowns;
- on cancellation, send `Z3_interrupt` for FFI sessions or terminate and restart a subprocess session if protocol synchronization cannot be guaranteed;
- retain the last inspected Djex progress and a separate solver progress.

8.6 Semantic deduplication before rendering

Djex already supports alpha equivalence and structural simplification. Add an optional semantic equivalence check:

$$\exists x.\ [\![f(x)]\!] \neq [\![g(x)]\!].$$

If unsatisfiable in the requested observation domain, put f and g in the same semantic class and present the syntactically smallest or best-rated representative. This can dramatically reduce Exference's repeated derivations and near-duplicates. The UI must say "equivalent under length/bag observations", not "extensionally equal", unless the encoding is exact for full values.

9 Leant integration in detail

9.1 Elaborate the contract in Lean

Leant's existing serializer is the model. Add a Lean metaprogram that receives the elaborated function type and contract, then emits a canonical semantic fragment. It should recognize an intentionally bounded vocabulary:

- propositions, equality, order, connectives, and quantifiers;
- `Nat`, `Int`, and fixed-width bit-vector operations;
- list/array/string measures and selected operations;
- constructors and non-dependent finite inductives;
- theorem-backed provider summaries registered by an attribute.

Anything else is returned as a structured refusal or opaque atom according to policy. As with the type serializer, pretty-printed source text is for diagnostics only; exact Lean names and elaborated structure are the authority.

A possible theorem-summary declaration is:

```
@[leant_synth_summary]
theorem List.reverse_length (xs : List alpha) :
    (List.reverse xs).length = xs.length := by
  simp
```

The attribute handler checks that the theorem matches a supported summary schema and serializes its exact constant name, quantified variables, preconditions, and equation. A stale or ill-shaped declaration fails summary registration rather than silently becoming an assumption.

9.2 Retain structured candidates in the engine

Replace or supplement:

```
SynthCandidates [[String]] [String]
```

with:

```
data SynthCandidate = SynthCandidate
  { synthExpression  :: Expression String
  , synthTypedView   :: Maybe LeantTypedCandidate
  , synthVariants    :: [String]
  , synthOrigin      :: SynthEngine
  , synthMetrics     :: CandidateMetrics
  }

data SynthOutcome
  = SynthCandidates [SynthCandidate] [String]
  | SynthRefuted RefutationStrength
  | SynthNoTerm [String]
```

This keeps today's rendering variants and scheduling logic while allowing semantic verification before or after rendering. The engine remains swappable because the structured candidate is still Djex-neutral rather than an Exference node.

9.3 Verification pipeline

For a proof-grade constrained query, process each candidate in this order:

1. **Lean type check.** Submit today's temporary example to the actual user environment. This catches renderer, namespace, implicit-argument, and universe errors.
2. **Z3 semantic check.** Encode the exact admitted contract fragment and ask for a counterexample. Reject `sat`; retain the model for explanation.
3. **Proof-plan generation.** Translate the symbolic derivation, candidate shape, and solver facts into a small Lean tactic plan.
4. **Lean theorem check.** Submit a temporary theorem binding the candidate and proving the contract. Accept proof-grade status only if Lean reports no errors or sorries.
5. **Binding.** Bind both candidate and theorem in the REPL's usual `it`-style order.

A generic generated checker is:

Listing 13: Final Leant behavioral checker shape

```
set_option autoImplicit true in
noncomputable example :
  let f : TargetType := candidate
  Contract f := by
    proof_plan
```

For simple definitional laws the plan is `rfl` or `simp`. For Presburger residuals it can be `simp [candidate, summaries] <;> omega`. For structural recursion it can be induction followed by simplification and arithmetic.

9.4 Proof-plan reconstruction tiers

The proof generator should be deliberately modest and compositional:

1. definitional reduction / `rfl`;
2. rewriting by exact registered summary theorems;
3. `simp` normalization;
4. `omega` for natural/integer linear arithmetic;
5. constructor congruence and extensionality;
6. induction on one structurally decreasing argument;
7. explicit case splits corresponding to candidate cases;
8. only later, imported proof certificates or richer tactic search.

Z3 models and unsat cores help choose splits and lemmas. They do not have to be translated into a general proof object. This avoids making Leant depend on the full stability and semantics of Z3's internal proof AST.

9.5 User-facing commands

A coherent command family could be:

```
:synth (f : List alpha -> List alpha)
  where forall xs, (f xs).length = xs.length

:set synth-contract-proof kernel|solver|bounded
:set synth-semantic-domain length,bag
:set synth-z3-timeout 1500
:set synth-counterexamples on
:set synth-semantic-dedupe on
```

Output should distinguish results explicitly:

```
1. fun xs => xs
   contract: proved by Lean kernel (rfl)

rejected: fun _ => []
   counterexample: input length = 1, output length = 0

search note: Exference queue pruned 247 nodes
solver note: 8 candidates checked, 5 counterexamples, 2 semantic duplicates
```

When the type search itself yields a Djinn refutation, Leant can still report it under its current safety rules. When a finite semantic grammar is exhausted, say "no satisfying program in grammar of size at most k using these providers", never "no term exists".

9.6 Proof-producing synthesis as a first-class feature

The strongest Leant feature is not merely filtering terms. It is synthesizing a pair:

$$(f,\ p) \quad \text{where} \quad p : \forall x,\ P(x) \to Q(x, f(x)).$$

The REPL can bind:

```
it1      : TargetType := ...
it1_spec : Contract it1 := by ...
```

This makes synthesized results reusable in later proofs, exportable to source, and independent of the solver after kernel checking. It also creates a natural feedback loop: newly proved summaries can be registered as providers for subsequent synthesis.

10 High-value features unlocked by the same integration

Behavioral term synthesis is the core, but several features become available almost for free once typed candidates, semantic summaries, and a persistent solver session exist.

10.1 Minimal counterexamples for every rejected candidate

A type-only synthesizer can say that a term has the requested type. A semantic synthesizer can say exactly why it is wrong. Z3 models should be decoded back through the semantic domain:

- list length, constructor skeleton, and selected provenance positions;
- concrete integers/bit-vectors and overflow flags;
- protocol state and event sequence;
- finite map entries or array indices on which outputs differ.

Then minimize the model. For arithmetic, optimize lexicographically by input size and absolute values. For ADTs, minimize constructor count. For sequences, minimize length and then lexicographic token values. A small counterexample is often more valuable to a user than a rejected/accepted bit.

Counterexamples can also seed regression tests automatically. Djex can emit QuickCheck or HUnit skeletons; Leant can emit an `example` showing the failing evaluation for computable candidates.

10.2 Unsat cores for impossible specifications

Name each user contract clause and each grammar restriction. If the selector synthesis query is unsatisfiable, request an unsat core. This can explain:

```
conflicting clauses:
  C2: output length = input length
  C5: output length = input length + 1

or, under the selected grammar:
  G3: direct argument use disabled
  G7: reverse provider disabled
  C1: output is a permutation of input
```

Cores are not necessarily minimal. Offer optional core minimization under a separate budget and label the result “small core” unless minimality is established.

10.3 Semantic equivalence clustering

Type-directed search often emits many syntactically different terms that are identical under the observation relevant to the user. Examples include eta variants, redundant case splits, different

compositions of length-preserving providers, or arithmetic expressions normalized to the same function.

Maintain a representative set. For a new candidate f, ask whether it differs from a representative g under the selected observations. If unsatisfiable, merge it into the class; otherwise retain the counterexample as a discriminator for future candidates. This is essentially an observational version-space data structure and can reduce both UI noise and repeated solver work.

10.4 Round-trip adapter and serializer synthesis

Given providers for two representations, synthesize:

$$\text{decode}(\text{encode}(x)) = x$$

or a partial round trip with explicit validity predicate. Applications include:

- AST/view adapters and compatibility shims;
- tuple/record reshaping;
- byte/bit packing with exact bit-vector semantics;
- normalization/denormalization pairs;
- protocol message encoders over bounded fields.

The typed grammar handles construction; Z3 handles bit-vector/arithmetic laws and counterexamples; Leant can certify the final pair when definitions are available.

10.5 Lens and update-law synthesis

For `get :: S -> A` and candidate `put :: S -> A -> S`, impose:

$$\begin{aligned} \text{GetPut}: &\quad \text{put}(s, \text{get}(s)) = s, \\ \text{PutGet}: &\quad \text{get}(\text{put}(s, a)) = a, \\ \text{PutPut}: &\quad \text{put}(\text{put}(s, a_1), a_2) = \text{put}(s, a_2). \end{aligned}$$

This is an excellent fit for Djex because the type drastically narrows the term grammar, while the laws remove the remaining absurd inhabitants. For records/tuples and finite ADTs, the encoding is exact and often quantifier-free after counterexample negation.

10.6 Bounds-safe arithmetic and bit-vector code

Types such as `Word32 -> Word32` are nearly unconstrained. Add laws about masks, range, monotonicity, inverse pairs, or no overflow. Z3's bit-vector theory gives exact machine semantics, unlike translating to unbounded integers and hoping overflow is irrelevant.

Examples:

- synthesize a field extractor/inserter satisfying round-trip and noninterference;
- synthesize alignment arithmetic with exact wraparound prohibition;
- synthesize index calculations under array-bound preconditions;
- infer the weakest simple precondition that prevents overflow.

This is one of the areas where the combined system could exceed conventional "type-to-term" tools decisively.

10.7 Protocol transition and state-machine synthesis

Represent a state as an algebraic datatype plus arrays/integers. A candidate transition function must preserve an invariant and satisfy event-specific postconditions. Models produce concrete bad traces; Fixedpoint can verify recursive reachability properties. Djex contributes type-safe construction from available transition primitives; Leant can turn the invariant proof into a theorem.

10.8 Contract inference

Run symbolic interpretation in the opposite direction. Given a candidate, infer a strong summary in a template family:

- affine relation $L_{out} = aL_{in} + b$;
- interval/range bounds;
- constructor-count equations;
- monotonicity or parity;
- bag inclusion/equality;
- bit-mask dependence.

Use models to fit parameters, then verify the inferred relation. In Leant, attempt to prove it. This gives a command such as `:laws term` that can explain what a synthesized or handwritten function does.

10.9 Precondition and invariant abduction

When a desired postcondition is false unconditionally, infer a precondition from a restricted template language. For example:

$$0 \le i < \text{length}(xs) \quad \Rightarrow \quad \text{lookup}(\text{update}(xs, i, v), i) = v.$$

CEGIS alternates between candidate preconditions and counterexamples. Optimize seeks a weak precondition by minimizing selected literals or thresholds. The result must be reported as template-relative; in Leant it becomes a theorem obligation.

10.10 Contract-aware provider ranking

A provider whose summary moves an abstract state toward the postcondition should be ranked above one that cannot help. Examples:

- under length preservation, prefer zero-delta providers;
- under sortedness, prefer providers known to return sorted sequences;
- under range bounds, penalize providers widening the interval;
- under round-trip laws, prefer known inverse pairs.

This does not require pruning. A cheap abstract distance can augment Exference's score and provider ratings, improving time to first satisfying candidate while keeping every branch available.

11 Soundness and semantic boundaries

11.1 Haskell is not a total set-theoretic language

The equation

$$\text{length}(f(xs)) = \text{length}(xs)$$

is ambiguous in lazy Haskell. Questions include:

- Is xs finite? Does its spine contain bottom?
- Does evaluating the left length terminate?
- May f inspect elements with `seq` even though length does not?
- Can f throw an exception, diverge, perform effects, or use unsafe coercion?
- Is equality observational equality, value equality, or equality in a total denotational fragment?

The system must state a semantic mode rather than hiding these choices.

11.1.1 Recommended Haskell modes

Mode	Meaning	Result label
`MathematicalTotal`	Values and providers belong to a total, pure, structural calculus. Datatypes are ordinary finite mathematical values.	Exact relative to checked semantic summaries.
`FiniteSpineTotal`	List/tree spine is finite and total; payloads may be opaque when the contract does not force them; candidate/provider capability set excludes strictness/unsafe/effects.	Exact for the stated spine observation.
`BoundedOperational`	Evaluate generated code on a finite domain/depth/timeout; catch exceptions and divergence according to policy.	Validated within explicit bounds; never “proved”.

Provider capabilities should be explicit:

```
data SemanticCapability
  = TotalStructural
  | MayInspectPayloadStrictly
  | MayDiverge
  | MayThrow
  | UsesEffects
  | UsesUnsafeFeatures
  | UsesTypeReflection
```

A proof-grade query rejects or downgrades providers whose capabilities are incompatible with the chosen domain. Parametricity-derived summaries are admitted only in a fragment where their premises hold.

11.2 Abstract interpretation can be sound without being complete

A length-only interpretation does not know list contents. It can nevertheless prove a length equation exactly if every admitted provider has an exact length transformer. For richer candidates, the interpretation may overapproximate:

$$\gamma([\![e]\!]^{\#}) \supseteq \{[\![e]\!](v) \mid v \in \gamma(input^{\#})\}.$$

Then:

- if the abstract bad-state query is unsatisfiable, the concrete bad state is impossible (sound proof);
- if it is satisfiable, the model may be spurious and must be concretized or refined before rejecting a candidate;
- a bounded abstract domain must never be reported as universal proof.

This suggests CEGAR for later versions: validate abstract counterexamples in a more precise domain or source-language evaluator, then refine the abstraction.

11.3 Trust in provider summaries

The formula Z3 proves is only as good as the asserted semantics. Result evidence should list every non-derived summary used by the candidate and contract. A possible trust ordering is:

1. constructor/measure facts derived from checked datatype declarations;
2. Lean-kernel theorem matching a supported summary schema;
3. independently checked certificate;
4. versioned builtin semantic axiom;
5. user assumption;
6. empirical observation.

A result's proof grade is the minimum grade of its dependencies. In Leant, final theorem checking can discharge the dependency regardless of how Z3 found the term; failed proof reconstruction simply means the result remains solver-assisted rather than kernel-proved.

11.4 Z3 statuses must remain three-valued

Never convert `unknown` to either acceptance or rejection by default. Preserve:

- reason unknown;
- timeout/interruption distinction;
- whether a model was returned despite unknown (such a model is not generally a proof or guaranteed to satisfy quantified assertions);

- the exact logic/options and solver version.

Policies may choose to show an unknown candidate after source-language checking, but the UI must say that the behavioral contract was not established.

11.5 What an “unsat” result proves

For a complete candidate, Z3 `unsat` proves:

> No valuation in the encoded semantic domain satisfies the asserted precondition and violates the encoded postcondition, assuming the asserted provider summaries and Z3’s correctness.

It does *not* by itself prove:

- arbitrary Haskell contextual equivalence;
- termination or exception freedom outside the admitted fragment;
- a theorem in Lean’s kernel;
- absence of satisfying programs outside a finite grammar;
- absence of a model when Z3 returned unknown.

11.6 Leant’s trust story

Leant can make Z3 entirely untrusted for accepted proof-grade results:

1. Djex proposes a term.
2. Z3 guides selection and provides candidate invariants/counterexamples.
3. Lean elaborates the exact term in the user’s environment.
4. Lean checks a theorem stating the contract.

A bug in the semantic encoder can cause missed candidates or wasted work, but it cannot make Lean accept a false theorem. This is the ideal asymmetry: solver completeness and encoder quality affect usefulness; the kernel protects correctness.

11.7 No-solution claims

Use precise wording:

Established fact	Permitted wording
Djinn complete refutation in a sound projection	“The translated type is constructively uninhabited” with current Leant caveats.
Finite typed grammar exhaustively encoded and Z3 returns unsat	“No satisfying term exists in this grammar”; include size/providers/domain.
All candidates from a truncated search were refuted	“No satisfying candidate found within the search bounds.”
Z3 returned unknown	“Solver could not determine the contract.”

Established fact	Permitted wording
Bounded evaluator found no failure	“Validated on the stated finite domain/bounds.”

12 Z3 integration engineering

12.1 Subprocess first, C API second

Djex and Leant already depend on Haskell’s `process` package. The lowest-friction MVP is a long-lived process:

```
z3 -in -smt2
```

The pinned Z3 shell explicitly supports SMT-LIB 2 input, standard input, soft and hard timeouts, models, and memory limits. This path avoids adding a platform-specific native Haskell binding while the semantic IR is still changing.

A later direct C API wrapper brings:

- structured AST construction without textual parsing;
- lower latency and allocation reuse;
- direct interrupt, model, core, Optimize, and Fixedpoint access;
- precise per-context ownership and statistics.

Keep both behind a small `SemanticSolver` interface and run differential tests. The subprocess backend remains useful as a reference implementation and deployment fallback.

12.2 Robust SMT-LIB session protocol

Do not spawn one process per candidate. A session manager should:

1. launch Z3 with an argument vector, never through a shell;
2. perform a version/capability handshake;
3. set deterministic seeds and output options;
4. maintain a declaration generation and push/pop depth;
5. attach a unique request ID and end sentinel using `echo`;
6. parse complete S-expressions, including quoted symbols and escaped strings;
7. enforce a solver timeout and a stricter host-side deadline;
8. on malformed output, unexpected EOF, or deadline ambiguity, kill and recreate the process rather than guessing whether the stream is synchronized;
9. capture stderr and recent commands in a bounded diagnostic ring buffer.

Representative transaction:

```
(push 1)
(assert (! (>= n 0) :named pre_nonnegative))
(assert (! (not (= outLen n)) :named violates_length))
```

```
(check-sat)
(get-info :reason-unknown)
(echo "|djex-request-000042-end|")
(pop 1)
```

Only issue `get-model` after `sat`; only issue `get-unsat-core` after `unsat` with production enabled. A protocol state machine should make invalid sequences unrepresentable.

12.3 C API ownership model

The C API uses opaque context-owned objects and explicit reference counting for solver, Optimize, and Fixedpoint objects. A Haskell FFI wrapper should enforce:

- one worker thread owns one Z3 context;
- ASTs never cross context boundaries;
- bracketed increments/decrements or a region that outlives every AST handle;
- cancellation invokes `Z3_interrupt` or the solver-local interrupt before waiting for worker termination;
- finalizers are a safety net, not the primary lifetime mechanism;
- callbacks never re-enter the same context unexpectedly.

A pool of independent contexts is simpler and safer than concurrent use of one context. Use structural normalized IR as the cross-worker currency.

12.4 Symbol encoding

Never derive semantic identity from pretty text alone. Create collision-free SMT symbols from stable IDs, with readable suffixes only for diagnostics:

```
|d42.list_length|
|p17.Data.List.reverse|
|h3.result|
|c5.contract_clause_length_preserved|
```

Keep a bidirectional table for model/core decoding. Quoted SMT symbols still require careful escaping of vertical bars and backslashes; centralize it and test it with property-based round trips.

12.5 Normalization and caching

Cache keys should include:

- alpha-normalized typed candidate;
- normalized contract and selected semantic domains;
- exact semantic inventory fingerprint and provider assignments;
- grammar bound/objectives for synthesis queries;
- semantic model version;
- Z3 version/build and relevant options;
- proof policy.

Useful caches:

- symbolic interpretation of a typed subexpression;
- candidate verdict and decoded counterexample;
- pairwise observational equivalence;
- CEGIS example sets and learned clauses;
- compiled base SMT context per semantic session.

Do not cache across an `unknown` unless the reason/options are included and the caller explicitly accepts it. Unsat cores and models should be treated as explanatory artifacts attached to a verdict, not as portable proof certificates.

12.6 Determinism and reproducibility

A constrained result should be reproducible from a diagnostic bundle containing:

- repository/build versions;
- normalized type query and contract;
- provider inventory and semantic summary fingerprints;
- search options and observed progress;
- solver logic/options/seeds;
- emitted SMT-LIB query or normalized solver IR;
- model/core/reason unknown;
- final Lean checker text when applicable.

Provider discovery order and Exference queue scheduling are already meaningful. Solver parallelism should preserve deterministic displayed order by default even if checks finish out of order.

12.7 Performance tactics

- Specialize the solver logic: use QF_LIA for length, QF_BV for machine code, and avoid general quantified SMT unless necessary.
- Simplify transfer functions in Haskell before encoding.
- Share declarations and contract context; candidate checks are push/pop deltas.
- Use assumptions to check sibling alternatives and extract reusable cores.
- Maintain discriminating counterexamples and test candidates on them before a full solver call.
- Run cheap source-level or abstract-domain checks before Z3.
- Batch semantic equivalence comparisons against representatives.
- Use incremental CEGIS and retain learned clauses across cost bounds where valid.
- Separate a fast interactive timeout from an explicit deep-search command.

13 Implementation sequence

The stages below are ordered by architectural leverage and risk, not by marketing impact. Each stage leaves a useful, testable system.

13.1 Stage A: semantic verification MVP

- Add pure semantic sorts, terms, formulas, contracts, summaries, and evidence.
- Implement length/size and linear integer domains for finite structural data.
- Add a typed-candidate compatibility projection for the first-order fragment.
- Implement a robust long-lived SMT-LIB process and S-expression parser.
- Filter complete Djinn/Exference candidates; return counterexamples.
- Add cache keys and exact result labels.
- Keep all existing unconstrained queries byte-for-byte compatible where possible.

The first end-to-end demonstrations should be:

1. `[a] -> [a]` preserving length;
2. `(a,b) -> (b,a)` satisfying two projection equations;
3. small integer/bit-vector functions satisfying range and round-trip laws.

13.2 Stage B: Leant proof-grade contracts

- Extend the Lean serializer with the contract fragment.
- Retain structured Djex expressions in `SynthOutcome`.
- Implement theorem-backed semantic summary registration.
- Generate and kernel-check simple proof plans: `rfl`, `simp`, `omega`, constructor cases.
- Bind both candidate and `itN_spec` theorem.
- Display counterexamples and solver/source/kernel status separately.

13.3 Stage C: semantic selection and explanation

- Semantic equivalence clustering.
- Minimal counterexamples.
- Unsat-core explanations for contract clauses and grammar restrictions.
- Contract-aware provider ranking.
- User-visible diagnostic bundle/export.

13.4 Stage D: typed sketch CEGIS

- Introduce a typed generated/sketch tree at the checked engine boundary.
- Generate finite alternatives for holes from lexical scope and inventory.
- Encode selectors, examples, blocking clauses, and cost objectives.

- Decode models to candidates and run the existing type/scope/render checks.
- Verify universally by existential counterexample search.
- Report exact finite-grammar exhaustion.

13.5 Stage E: Exference semantic oracle

- Refactor the pure search into explicit state/step/expansion APIs.
- Define branch abstractions and necessary feasibility conditions.
- Add batched oracle checks and semantic lower bounds to ranking.
- Preserve an oracle-disabled path and compare candidate order/results.
- Prove by construction that only unsatisfiable necessary conditions prune.

13.6 Stage F: recursive invariant synthesis

- Relational/CHC semantics for structurally recursive candidate forms.
- Fixedpoint/PDR integration and invariant extraction.
- Ranking/decrease templates.
- Lean induction proof plans seeded by discovered invariants.
- CEGAR between measure, shape, and full ADT domains.

13.7 Concrete file-level changes

Area	Change
`Djex/synthesis/.../Generated.hs`	Keep source-compatible; add projection hooks or companion typed generated module.
`Djex/synthesis/.../Candidate.hs`	Optionally expose an elaborated-candidate wrapper; do not burden every existing candidate.
`Djex/synthesis/.../Selection.hs`	Reuse monadic fold; add constrained selection policies in a new module.
`Djex/synthesis/.../Query.hs`	Leave `QueryEvidence` semantics unchanged; new semantic result wrapper only.
`Djex/djinn/.../ProofToGenerated.hs`	Emit typed projection before erasure.
`Djex/exference/.../Internal/Candidate.hs`	Emit typed projection from the independently checked internal term.
`Djex/exference/.../ExferenceNode.hs`	Later expose stable partial-sketch projection and semantic abstraction.
New `Djex/synthesis/.../Semantic/*`	Pure contract, summary, symbolic, and evidence foundation.
New `Djex/z3/...` or package djex-z3	Solver transport, encoding, models, cores, Optimize, Fixedpoint.
`Leant/src/Leant/Synth/Fragment.hs`	Add contract and summary serializers; keep Lean-side elaboration authoritative.

Area	Change
`Leant/src/Leant/Synth/Engine.hs`	Preserve structured candidates and invoke semantic layer behind the narrow boundary.
`Leant/src/Main.hs`	Add commands, semantic checking, proof-plan checking, and candidate/spec bindings.
New `Leant/src/Leant/Synth/ProofPlan.hs`	Generate small auditable Lean proof scripts.

14 Testing and evaluation

14.1 Unit and property tests

- Sort checking, binder scope, substitution, alpha normalization, and formula normalization for the semantic IR.
- SMT symbol and S-expression parser round trips, including quoted identifiers, strings, numerals, algebraic values, arrays, datatypes, and error responses.
- Golden SMT-LIB transcripts for each theory/domain.
- Mock-server tests for partial writes, split reads, stderr noise, crash, timeout, interrupted check, malformed response, and stale sentinel.
- Derived datatype measure equations checked against small exhaustive values.
- Symbolic interpreter compared with a concrete evaluator on every admitted primitive and generated small expression.
- Cache-key invariance under alpha renaming and sensitivity to summary/provider changes.
- No semantic feature changes unconstrained query evidence, progress, candidate validation, or lazy consumption.

14.2 Differential tests

Use at least three independent paths where possible:

1. brute-force enumeration over small finite domains;
2. SMT-LIB subprocess backend;
3. C API backend, once implemented;
4. Lean theorem checking for proof-grade Leant cases.

A solver `unsat` that disagrees with brute force is a critical encoder bug. A Lean proof failure after solver `unsat` is also a valuable signal: either proof reconstruction is incomplete or the semantic assumptions exceed what was stated in Lean.

14.3 Metamorphic tests

- Alpha-renaming locals and type variables preserves verdicts.
- Adding an unused provider does not change a complete candidate's semantics.
- Strengthening a precondition cannot create a new counterexample.

- Strengthening a postcondition cannot turn a previously refuted candidate into a proved one.
- Increasing grammar size cannot invalidate an already found candidate.
- Reordering semantically identical alternatives changes neither decoded program set nor proof status, only deterministic tie-breaking.
- Replacing a provider summary by an equivalent theorem leaves fingerprints different but verdicts equal.

14.4 Regression corpus

Create a versioned corpus by domain:

- list length/bag/order transformations;
- products, sums, records, lenses, and adapters;
- finite ADT constructor-count laws;
- natural/integer arithmetic with preconditions;
- exact bit-vector packing and overflow;
- small protocol state machines;
- recursive map/fold/filter variants with induction proofs;
- deliberately unsupported Haskell strictness/unsafe cases;
- Leant universe, implicit-argument, namespace, and theorem-summary cases.

Every fixed bug should add both a positive and a nearby negative case when possible.

14.5 Performance measurements

Measure more than total time:

Metric	Why it matters
Time to first type-correct candidate	Baseline engine behavior.
Time to first contract-satisfying candidate	Primary user latency.
Candidates generated / solver-checked / rejected	Selectivity and wasted search.
Counterexamples reused before solver	CEGIS/cache effectiveness.
Solver calls and median/p95 duration	Transport and encoding overhead.
Semantic classes vs syntactic candidates	Deduplication value.
Branches expanded/pruned by reason	Effect of deep oracle integration.
Cache hit rate by layer	Session reuse and normalization quality.
Lean proof-plan success rate	Gap between solver verification and kernel proof.

Metric	Why it matters
Peak Exference queue and memory	Whether pruning changes practical scalability.

Benchmarks should compare: no semantic layer, post-filter only, contract-aware ranking, typed CEGIS, and branch pruning. A feature is successful when it improves time to a proof-grade result, not merely when it increases solver activity.

15 Risks and design traps

Do not translate arbitrary source language semantics in the first version

A general Haskell-to-SMT or Lean-to-SMT translator would dominate the project and blur its trust claims. Start with an explicit total semantic fragment, exact measures, and refusal outside it. Add domains incrementally.

Do not hide solver IO inside pure search

Exference's purity is valuable. Expose a stepping boundary or use a pure formula builder with an outer driver. `unsafePerformIO` would make cancellation, repeatability, and error handling unreliable.

Do not confuse candidate proof with search completeness

A candidate can be proved correct while the search is truncated. Conversely, every seen candidate can fail while a satisfying one remains unexplored. Preserve both facts.

Do not treat provider names as semantics

A function named `reverse`, `sort`, or `length` has no solver meaning without an exact checked summary tied to that declaration and environment.

Do not call bounded testing a theorem

Finite enumeration is excellent for finding counterexamples and validating encoders. Its positive result is bounded evidence unless a finite domain is the complete stated domain.

Do not make proof reconstruction all-or-nothing

Leant should still show a solver-verified/type-checked candidate when configured to do so if its small proof-plan generator cannot close the theorem. Proof status is a tier, not a reason to discard useful search output silently.

Do not optimize before establishing a stable semantic IR

A fast FFI, custom tactic, or deep search hook will not rescue an ambiguous contract or untyped translation boundary. Build the checked neutral representation and evidence model first.

16 Final recommendation

The combination is unusually promising because the three projects cover distinct parts of the problem with unusually compatible boundaries:

- Djex supplies typed logical construction, two complementary search engines, checked inventories, explicit progress/evidence, and a shared generated AST.
- Leant supplies source-language elaboration, a live theorem environment, provider discovery, and a kernel that can remove Z3 from the trusted base.
- Z3 supplies fast decision procedures, countermodels, cores, optimization, and recursive invariant machinery.

The best first implementation is not to replace Djinn or Exference with Z3. It is to add a semantic refinement loop around them. Start by post-filtering complete typed candidates in the length/size/linear arithmetic fragment, with exact counterexamples and honest evidence. In parallel, make Leant synthesize and kernel-check the companion contract theorem. Once that substrate is stable, typed sketch CEGIS and Exference branch pruning become natural extensions rather than speculative rewrites.

The resulting feature is qualitatively different from traditional type inhabitation:

From "this term has the requested shape"

to "this term has the requested shape, satisfies the requested law, comes with a minimal counterexample when it does not, and can carry a kernel-checked proof when used through Leant."

That is a credible path to a genuinely new synthesis tool rather than a thin solver integration.

A Proposed Haskell API in more detail

This appendix sketches a public API shape sufficiently concrete to expose hidden design conflicts. Names are provisional.

A.1 Semantic IR

```haskell
data Sort
  = SBool
  | SInt
  | SBitVector Natural
  | SUninterpreted SemanticTypeId
  | SDatatype SemanticDatatypeId [Sort]
  | SSequence Sort
  | SArray Sort Sort
  deriving (Eq, Ord, Show)

data Term variable
  = TVar variable Sort
  | TBool Bool
  | TInt Integer
  | TBV Natural Integer
  | TApp FunctionId [Term variable] Sort
  | TIte (Formula variable) (Term variable) (Term variable)
  | TConstructor ConstructorId [Term variable] Sort
```

```
  | TSelector SelectorId (Term variable) Sort
  | TSeqEmpty Sort
  | TSeqUnit (Term variable)
  | TSeqConcat [Term variable]
  | TSelect (Term variable) (Term variable) Sort
  | TStore (Term variable) (Term variable) (Term variable)
  deriving (Eq, Ord, Show)

data Formula variable
  = FTrue
  | FFalse
  | FNot (Formula variable)
  | FAnd [Formula variable]
  | FOr [Formula variable]
  | FImplies (Formula variable) (Formula variable)
  | FEq (Term variable) (Term variable)
  | FLt (Term variable) (Term variable)
  | FLe (Term variable) (Term variable)
  | FPredicate PredicateId [Term variable]
  deriving (Eq, Ord, Show)
```

The initial public IR can omit quantifier constructors because candidate verification uses free input constants. Quantifiers needed by summary axioms or advanced contracts can remain in an internal checked representation until trigger/domain policy is clear.

A.2 Checked contracts

```
data ContractSource ty name = ContractSource
  { sourceInputs        :: [(name, ty)]
  , sourceResultName    :: name
  , sourcePrecondition  :: SourceFormula name
  , sourcePostcondition :: SourceFormula name
  , sourceDomain        :: SemanticDomain
  , sourceObjectives    :: [Objective]
  }

data ContractError ty name
  = DuplicateContractBinder name
  | UnboundContractVariable name
  | ContractSortMismatch Sort Sort
  | ContractResultTypeMismatch ty ty
  | UnknownMeasure Name
  | MeasureNotDefinedAtType Name ty
  | UnsupportedContractOperator SourceSpan
  | DomainIncompatibleProvider Name SemanticCapability

mkCheckedContract
  :: CheckedSemanticEnvironment ty
  -> QueryRequest ty options
  -> ContractSource ty name
  -> Either (ContractError ty name) CheckedContract
```

The constructor of `CheckedContract` should be private. Its normalized form can be hashed, encoded, and combined with a typed candidate without repeating name/sort checks.

A.3 Symbolic interpretation

```
data SymbolicValue variable
  = ScalarValue (Term variable)
  | DatatypeValue SemanticDatatypeId [Sort] (Term variable)
  | AbstractValue (Map ObservationId (Term variable))

data InterpretationError
  = UnsupportedExpression NodePath String
  | MissingProviderSummary Name ObservationId
  | SummaryTypeMismatch Name
  | DomainViolation SemanticCapability
  | RecursiveCandidateRequiresFixedpoint
  | ResidualConstraintUnsupported String

interpretCandidate
  :: CheckedSemanticEnvironment ty
  -> CheckedContract
  -> TypedExpression ty local
  -> Either InterpretationError (SymbolicFunction local)
```

An `AbstractValue` is a product of observations, for example length plus bag. Composition requires only the provider clauses for requested observations.

A.4 Verdicts and counterexamples

```
data Counterexample = Counterexample
  { counterexampleInputs    :: Map BinderId SemanticValue
  , counterexampleResult    :: Maybe SemanticValue
  , violatedClauses         :: [ContractClauseId]
  , counterexampleMinimality :: MinimalityStatus
  , rawModelFingerprint     :: Fingerprint
  }

data UnknownReason
  = SolverReturnedUnknown Text
  | SolverTimedOut Duration
  | SolverInterrupted
  | SolverProtocolFailure Text
  | UnsupportedSemantics InterpretationError
  | ProofReconstructionFailed Text

data SolverProgress = SolverProgress
  { solverChecks        :: Natural
  , solverSat           :: Natural
  , solverUnsat         :: Natural
  , solverUnknown       :: Natural
  , solverCacheHits     :: Natural
  , solverElapsed       :: NominalDiffTime
  }
```

A.5 Sketches

```
data Sketch ty local = Sketch
  { sketchRoot       :: TypedExpression ty local
  , sketchHoles      :: Map HoleId (HoleAlternatives ty local)
  , sketchCostModel  :: CostModel
  , sketchGrammarKey :: GrammarFingerprint
```

```
  }

data HoleAlternatives ty local = HoleAlternatives
  { holeType         :: ty
  , holeScope        :: CheckedScope ty local
  , holeChoices      :: NonEmpty (TypedExpression ty local)
  }

data SynthesisOutcome candidate
  = SynthesisFound candidate SemanticVerdict
  | SynthesisGrammarUnsat GrammarExhaustion UnsatExplanation
  | SynthesisUnknown UnknownReason
  | SynthesisCancelled
```

The finite alternative vector makes model decoding total. A malformed selector model is an internal protocol error, not a user-level unsupported case.

B Complete SMT-LIB sketch for the length example

The following illustrates a finite selector grammar rather than an optimized production encoding. The candidate choices are identity, empty, duplicate, and totalized tail. The selector is chosen to satisfy current examples, then the decoded candidate is verified separately.

Listing 14: Selector synthesis over finite examples

```
(set-logic QF_LIA)
(set-option :produce-models true)

; 0 = id, 1 = empty, 2 = duplicate, 3 = tail0
(declare-const choice Int)
(assert (and (<= 0 choice) (< choice 4)))

(define-fun out-len ((n Int)) Int
  (ite (= choice 0) n
  (ite (= choice 1) 0
  (ite (= choice 2) (* 2 n)
                    (ite (= n 0) 0 (- n 1))))))

; Current CEGIS examples.
(assert (= (out-len 0) 0))
(assert (= (out-len 1) 1))
(assert (= (out-len 2) 2))

(check-sat)
(get-value (choice))
```

A model selects `choice = 0`. Verification is a fresh query:

Listing 15: Universal verification as counterexample search

```
(set-logic QF_LIA)
(set-option :produce-models true)

(declare-const n Int)
(assert (>= n 0))

; Decoded choice 0: identity.
(define-fun out-len () Int n)
```

```
(assert (not (= out-len n)))

(check-sat)
; unsat: no finite nonnegative input length violates the law.
```

For a grammar with nested holes, selector variables determine a DAG or tree. Production encodings should add acyclicity/level constraints, share common subterms intentionally, and compute cost with integer expressions. Enumerating all models uses a blocking clause:

```
(assert (not (and (= choice0 2) (= choice1 0) (= choice2 4))))
```

Prefer a smaller learned clause when the counterexample depends on only a subset of the choices. Assumption literals and unsat cores can identify such subsets.

C Generated Lean checker examples

C.1 Definitional candidate

```
set_option autoImplicit true in
noncomputable example :
  let f : List alpha -> List alpha := fun xs => xs
  forall xs, (f xs).length = xs.length := by
  intro xs
  rfl
```

C.2 Summary-theorem candidate

```
set_option autoImplicit true in
noncomputable example :
  let f : List alpha -> List alpha := List.reverse
  forall xs, (f xs).length = xs.length := by
  intro xs
  simp [f]
```

C.3 Structural recursive candidate

A schematic rotation definition and proof shape:

```
set_option autoImplicit true in
noncomputable example :
  let f : List alpha -> List alpha := fun xs =>
    match xs with
    | []      => []
    | x :: xt => xt ++ [x]
  forall xs, (f xs).length = xs.length := by
  intro xs
  cases xs with
  | nil => rfl
  | cons x xt =>
      simp [f]
```

The actual generated term may require fully qualified names, explicit universe arguments, or binder variants. Leant should continue to use its current renderer and verify every exact spelling

in the active environment.

C.4 Arithmetic residual closed by omega

```
set_option autoImplicit true in
noncomputable example :
  let f : Nat -> Nat := fun n => (n + 1) - 1
  forall n, f n = n := by
  intro n
  simp [f]
```

For more involved linear obligations, use:

```
  simp [f, registered_summary_1, registered_summary_2]
  omega
```

The generated script is advisory until Lean accepts it. Leant should retain the failed script and goal state in diagnostics to improve proof-plan coverage.

D Theory and feature matrix

Feature	Preferred Z3 theory/API	Djex contribution	Leant proof route
Length/size laws	QF_LIA, models	Typed terms; measure summaries; search	`simp`, `omega`, induction.
Permutation/bag laws	Arrays + LIA, or provenance abstraction	Parametric grammar; constructor transfer	Extensionality, count lemmas, induction.
Order-sensitive list laws	Sequences, ADTs	Typed provider composition	Sequence/list lemmas, induction.
Products, records, lenses	ADTs, EUF	Constructor and projection inhabitation	`rfl`, cases, extensionality.
Bit packing	QF_BV, Optimize	Type-safe composition of bit operations	Bit-vector lemmas, `decide`/native tactics where available.
Bounds-safe indexing	LIA/BV + arrays	Provider and precondition synthesis	`omega`, array/list lemmas.
Round trips	ADTs/BV/arrays	Paired typed sketches	Rewriting and extensionality.
Protocol transitions	ADTs, arrays, Fixedpoint	Transition grammar and provider inventory	Invariant theorem, induction/reachability lemmas.
Recursive invariants	Fixedpoint/PDR, recursive definitions	Structural recursive candidate forms	Generated induction with discovered invariant.
Semantic deduplication	Any selected observation theory	Candidate stream and metrics	Optional theorem of equivalence.
Counterexample minimization	Optimize	Decode to source-level values	Optional computable failing example.
Conflicting contracts	Tracked assertions, unsat cores	Clause IDs and grammar restrictions	Explain before proof attempt.
Contract inference	Models + verification + Optimize	Candidate and summary templates	Prove inferred law.

Feature	Preferred Z3 theory/API	Djex contribution	Leant proof route
Precondition abduction	CEGIS + Optimize	Template grammar	Prove conditional theorem.

E Source map

The report's repository-specific conclusions are based on the following files at the pinned revisions. Links are immutable commit URLs.

Djex

- Repository overview and public architecture: `Djex/README.md`
- Shared synthesis foundation overview: `Djex/synthesis/README.md`
- Query requests, evidence, and checked result envelope: `Djex/.../Synthesis/Query.hs`
- Operational progress and truncation: `Djex/.../Synthesis/Search.hs`
- Candidate envelope: `Djex/.../Synthesis/Candidate.hs`
- Scope-aware generated terms and rewrites: `Djex/.../Synthesis/Generated.hs`
- Streaming and ranked selection: `Djex/.../Synthesis/Selection.hs`
- Neutral declarations: `Djex/.../Synthesis/Declaration.hs`
- Checked Djinn adapter: `Djex/.../Djex/Djinn.hs`
- Checked Exference adapter: `Djex/.../Djex/Exference.hs`
- Exference search node and typed-hole state: `Djex/.../Internal/ExferenceNode.hs`
- Exference search implementation: `Djex/.../Internal/Exference.hs`
- Cabal package/dependency structure: `Djex/djex.cabal`

Leant

- User-facing behavior and safety rules: `Leant/README.md`
- Djex engine boundary and candidate merging/rendering: `Leant/src/Leant/Synth/Engine.hs`
- Lean-side goal fragment serializer and candidate checker generator: `Leant/src/Leant/Synth/Fragment.hs`
- REPL verification loop, tactic suggestions, and candidate checking: `Leant/src/Main.hs`
- Package structure: `Leant/leant.cabal`

Z3

- Repository overview and supported bindings: `z3/README.md`
- Core C API: sorts, solvers, models, proofs, recursive functions, interrupts: `z3/src/api/z3_api.h`
- Optimization API: `z3/src/api/z3_optimization.h`
- Fixedpoint/PDR API: `z3/src/api/z3_fixedpoint.h`

- Command-line SMT-LIB/standard-input/resource options: `z3/src/shell/main.cpp`
- Sequence implementation/API surface: `z3/src/api/api_seq.cpp`

F One-page implementation checklist

Definition of a successful first release

1. Existing unconstrained Djex and Leant synthesis behavior remains unchanged when the semantic feature is disabled.
2. A checked contract for a finite-spine list length law can be elaborated in both a Djex frontend and Leant.
3. Complete typed candidates can be interpreted in an exact length domain.
4. A persistent Z3 session returns counterexamples for bad candidates and `unsat` for good ones, with cancellation, timeout, and protocol recovery.
5. Search progress and semantic verdicts remain separate in the API and output.
6. Leant can kernel-check at least identity, reverse, and a case-based rotation together with their length theorems.
7. Every accepted result lists its semantic domain and summary trust; every rejected result can show a small counterexample.
8. Differential tests compare solver verdicts with exhaustive small-domain evaluation, and Leant proof-grade cases contain no `sorry`.

End of report. The TeX source is intentionally self-contained and can be rebuilt with XeLaTeX or `latexmk -xelatex`.